

AUTOENCODER TASK

-VIDIT ARORA(25/A13/058)

Image compression and reconstruction task

Implement an autoencoder model on the given Image Dataset.

Goals and constraints

1. Build the model from scratch using PyTorch. Do not use pre-trained models or existing notebooks.
2. Experiment with at least two different backbone designs each for :
 - a) Artificial Neural Networks (fully connected)
 - b) Convolutional Neural Networks
(Vary architecture design and bottleneck size)
3. Visualize input images and their reconstructed outputs. Observe how images are encoded in the latent space.
4. Plot and compare training and validation loss curves for different models.

Define the autoencoder pipeline clearly :

1. Your Input: Image
2. Encoder : Compresses the image into a latent representation
3. Decoder : Reconstructs the image from the latent representation
4. Output: Reconstructed image
5. Train the model to minimize the difference between input and reconstructed output (Re-construction Loss)

Pre-requisites :

1. Structure of a CV Model (Encoder, Neck, Decoder)
2. Latent space and representation learning
3. Autoencoders and their variants

Your Dataset :

<https://share.google/rgh2tiRtjlfipWuTR>

Suggested bottleneck size : 32-64

1. Experimenting with ANN encoder/decoder

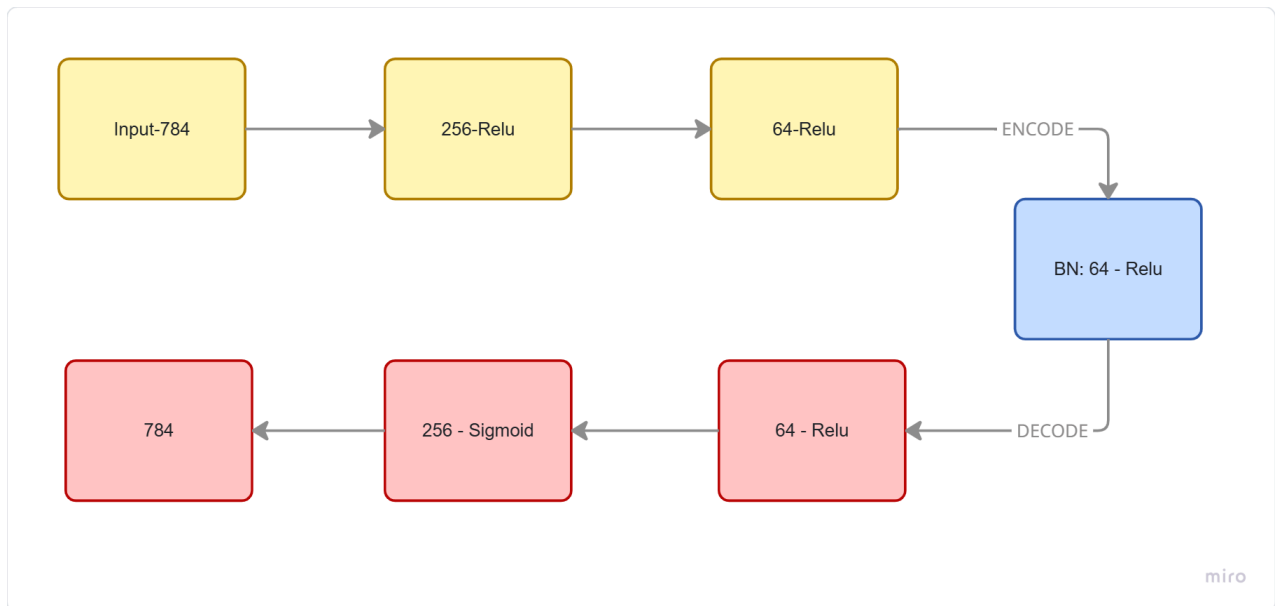
Approach: First of all, one will hardly see ANNs getting used in these tasks and i got to know the reason very well after implementing them along with CNNs , so here i tried two extreme models that is one shallow network with bottle neck 64(more space to store information, but lacks the complex layers to understand deep patterns) and another a bit deep with bottle neck 32 with batch norm and dropout applied(smarter, deeper network with a tight bottle neck),

I chose these designs to test if a deeper architecture could compensate for a smaller bottleneck. ANN-v1 was a basic with high capacity (64 units), while ANN-v2 was a more advanced deep pipeline with only 32 units.

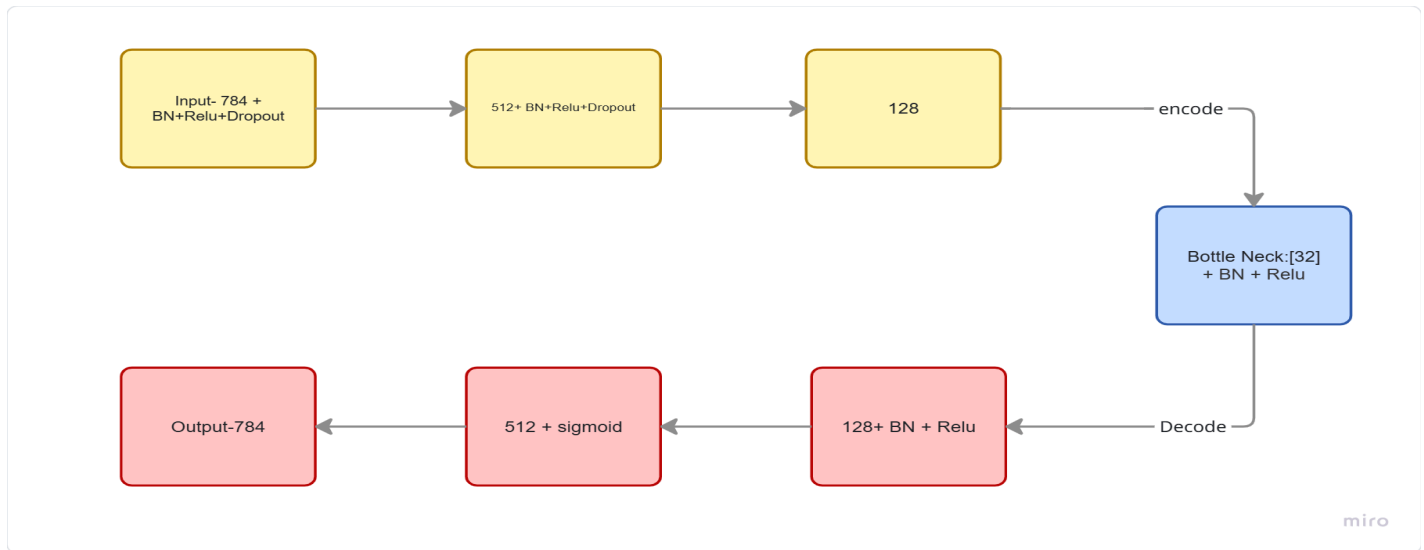
Learning: The comparison proved that adding layers like Batch Normalization allows the model to learn a more efficient representation, matching the quality of the wider model with half the space,plus v2 had less number of parameters than v1 still performed better, this tells that efficiency matters

Observation: Both ANNs produce blurry letters. They are good at getting the general shape but bad at sharp edges.

ANN-V1:



ANN-V2:



2. Experimenting with CNN encoder/decoder

Approach: For CNNs I initially tried 3 models, first was a simple 2-layer encoder/decoder. I gave it a larger bottleneck to see if basic convolutions could beat the deep ANN, second was a deeper model with LeakyReLU and more filters, but a tighter bottleneck. This was to see if Depth + Convolutions could compress the image into just 32 numbers without losing the letter's identity, both of these did not have skip connections so I tried a third with a U-net style skip connections to pass extreme details directly from the encoder to the decoder, bypassing the bottleneck.

Learnings: As expected each of the CNN models gave better results than both the ANN models as these utilise spatial features and “know” that pixels are neighbours, CNN-v3 gave me a near-zero loss (0.000034) and perfect images, but its PCA/t-SNE plots were nearly blank. I realized that skip connections create a type of shortcut. The model stopped using the bottleneck to learn the meaning of the letter because it could just copy-paste pixels through the skips, also when I tried latent interpolation on CNN-v3, I got checkered gray boxes. This made me realise that skip connections are great for making (exact copies, but they are bad for creating new ones because the bottleneck

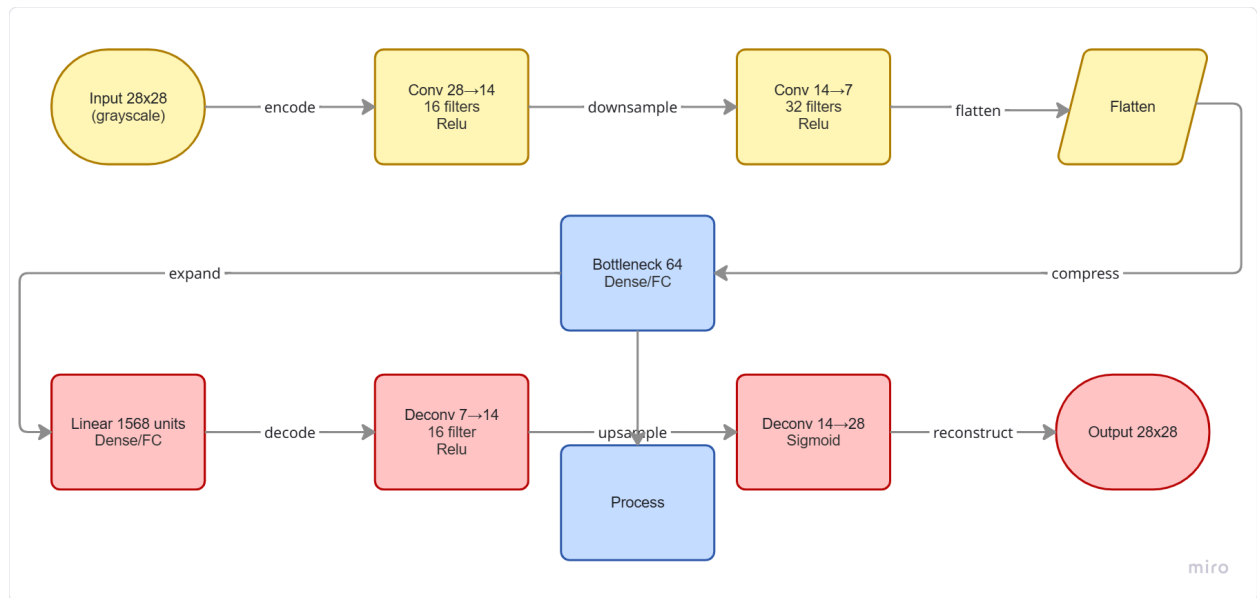
becomes "empty" or "lazy." And also CNNs used fewer parameters than the wide ANNs but produced much sharper edges and recognizable handwriting, so they are more efficient too.

Observations: CNN-v1 performed better than the deeper CNN-v2. This taught me that a larger bottleneck (64 vs 32) often provides a bigger performance boost than simply adding more layers.

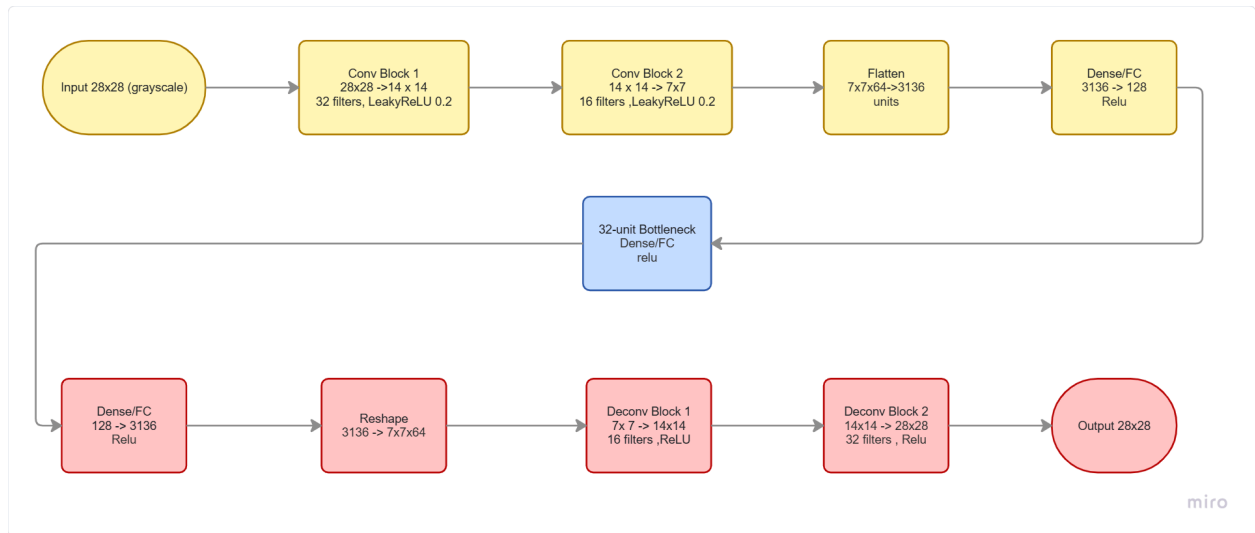
Also at the end of the notebook i tested Cnns with architecture of CNN-v1 with different bottle necks considering

16,32,48,64,128 , and observation had was initially from jumping 16 to 32 significant reduces the loss compared to later stages like 48 or 64 after which loss kind of saturates and adding more space to the bottleneck doesn't actually help the model. It just wastes memory. The lines for 64 and 128 are almost on top of each other, it proves that 64 is the optimal size.\

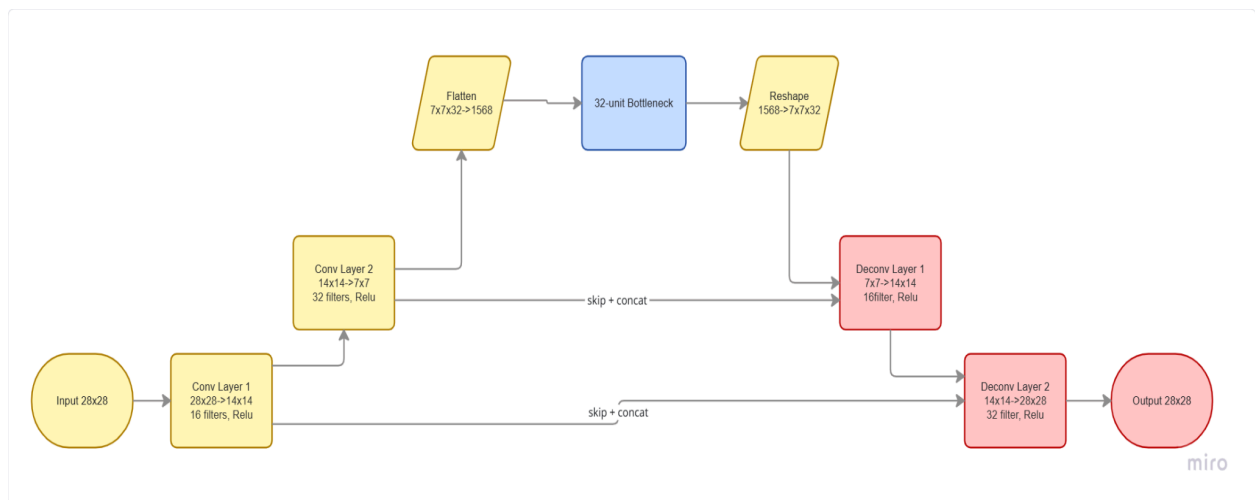
CNN-v1:



CNN-v2:



CNN-v3:



Dimensionality Reduction: In order to visualise the 32 or 64 dimensional latent space we had to use PCA and t-Sne that reduced dimensions to 2D , PCA struggles with non linear data that is where t-SNE comes into play

Latent Space Interpolation: Instead of just reconstructing an image, I took the code for Letter A and the code for Letter B and mathematically walked from one to the other. This tests if the latent space is continuous. If the middle images look like a smooth (a hybrid letter), means the model has learned the "geometry" of handwriting. : As I mentioned before, the checkered pattern in v3 interpolation proved that Skip Connections break the generative power of the bottleneck.